Expt. No. _____ Date: _____

ASSIGNMENT 1

AIM: To Implement AND, OR and NOT Gates Using VHDL

AND Gate

Theory: AND gate is a basic digital logic gate that performs logical multiplication. It produces a HIGH (1) output when ALL inputs are HIGH (1). If any one of the inputs is LOW (0), the output becomes LOW (0). It is commonly used in control and decision-making circuits. **Expression: Y = A.B**

Truth Table:

A	B	Y = A.B
0	0	0
0	1	0
1	0	0
1	1	1

VHDL Code:

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity and1 is
Port (inA1 : in STD_LOGIC; -- AND GATE INPUT
inA2 : in STD_LOGIC; -- AND GATE INPUT
OA : out STD_LOGIC); -- AND GATE OUTPUT
end and1;
architecture dataflows of and1 is
begin
OA <= inA1 and inA2; -- LOGIC OF 2 INPUT AND GATE
end dataflows;
```

Teacher's Signature: _____

Expt. No. _____ Date: _____

OR Gate

Theory: OR gate is a fundamental digital logic gate that performs logical addition. It produces a HIGH (1) output if at least one of the inputs is HIGH (1). The output becomes LOW (0) only when all inputs are LOW (0). It is widely used in signal combining and decision circuits. **Expression: Y = A+B**

Truth Table:

A	B	Y = A+B
0	0	0
0	1	1
1	0	1
1	1	1

VHDL Code:

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity or1 is
Port (inA1 : in STD_LOGIC; -- OR GATE INPUT
inA2 : in STD_LOGIC; -- OR GATE INPUT
OA : out STD_LOGIC); -- OR GATE OUTPUT
end or1;
architecture dataflows of or1 is
begin
OA <= inA1 or inA2; -- LOGIC OF 2 INPUT OR GATE
end dataflows;
```

NOT Gate

Theory: NOT gate, also an inverter, is a digital logic gate that reverses the input signal. It produces the opposite of the given input. If the input is HIGH (1), the output is LOW (0) and vice versa. It is mainly used for signal inversion operations. **Expression: Y = A'**

Truth Table:

A	Y = A'
0	1
1	0

VHDL Code:

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity not1 is
```

Teacher's Signature: _____

```
Port (inA1 : in STD_LOGIC; -- NOT GATE INPUT
OA : out STD_LOGIC); -- NOT GATE OUTPUT
end not1;
architecture dataflows of not1 is
begin
OA <= not inA1; -- LOGIC OF 1 INPUT NOT GATE
end dataflows;
```

Expt. No. _____ Date: _____

ASSIGNMENT 2

AIM: To Implement NAND, NOR, XOR, XNOR Gates Using VHDL

NAND Gate

Theory: NAND gate is a universal gate. Combination of AND + NOT. Output is LOW (0) only when ALL inputs are HIGH (1). **Expression: Y = (A.B)'**

Truth Table:

A	B	Y=(A.B)'
0	0	1
0	1	1
1	0	1
1	1	0

VHDL Code:

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity nand1 is
Port (inA1 : in STD_LOGIC; -- NAND GATE INPUT
inA2 : in STD_LOGIC; -- NAND GATE INPUT
OA : out STD_LOGIC); -- NAND GATE OUTPUT
end nand1;
architecture dataflows of nand1 is
begin
OA <= inA1 nand inA2; -- LOGIC OF 2 INPUT NAND GATE
end dataflows;
```

NOR Gate

Theory: NOR gate is also a universal gate. Combination of OR + NOT. Output is HIGH (1) only when ALL inputs are LOW (0). **Expression: Y = (A+B)'**

Truth Table:

A	B	Y=(A+B)'
0	0	1
0	1	0
1	0	0
1	1	0

VHDL Code:

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;
```

Teacher's Signature: _____

Expt. No. _____ Date: _____

```
Entity nor1 is
Port (inA1 : in STD_LOGIC; -- NOR GATE INPUT
inA2 : in STD_LOGIC; -- NOR GATE INPUT
OA : out STD_LOGIC); -- NOR GATE OUTPUT
end nor1;
architecture dataflows of nor1 is
begin
OA <= inA1 nor inA2; -- LOGIC OF 2 INPUT NOR GATE
end dataflows;
```

Teacher's Signature: _____

Expt. No. _____ Date: _____

XOR Gate

Theory: XOR (Exclusive-OR) gives HIGH (1) when inputs are different. Output is LOW (0) when both inputs are same. **Expression: Y = A XOR B = A'B + AB'**

Truth Table:

A	B	Y=A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

VHDL Code:

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity xor1 is
Port (inA1 : in STD_LOGIC; -- XOR GATE INPUT
inA2 : in STD_LOGIC; -- XOR GATE INPUT
OA : out STD_LOGIC); -- XOR GATE OUTPUT
end xor1;
architecture dataflows of xor1 is
begin
OA <= inA1 xor inA2; -- LOGIC OF 2 INPUT XOR GATE
end dataflows;
```

XNOR Gate

Theory: XNOR (Exclusive-NOR) is complement of XOR. Output is HIGH (1) when both inputs are same. Also called equivalence gate. **Expression: Y = (A XOR B)' = AB + A'B'**

Truth Table:

A	B	Y=A XNOR B
0	0	1
0	1	0
1	0	0
1	1	1

VHDL Code:

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity xnor1 is
Port (inA1 : in STD_LOGIC; -- XNOR GATE INPUT
```

Teacher's Signature: _____

```
inA2 : in STD_LOGIC; -- XNOR GATE INPUT
OA : out STD_LOGIC); -- XNOR GATE OUTPUT
end xnor1;
architecture dataflows of xnor1 is
begin
OA <= inA1 xnor inA2; -- LOGIC OF 2 INPUT XNOR GATE
end dataflows;
```

Expt. No. _____ Date: _____

AIM: To Implement NAND, NOR, XOR, XNOR Gates Using Basic Gates

NAND Using Basic Gates (AND + NOT)

NAND = AND gate output fed into NOT gate.

Step 1: Y1 = A.B (AND gate)

Step 2: Y = (Y1)' = (A.B)' (NOT gate)

Expression: Y = (A.B)'

NOR Using Basic Gates (OR + NOT)

NOR = OR gate output fed into NOT gate.

Step 1: Y1 = A+B (OR gate)

Step 2: Y = (Y1)' = (A+B)' (NOT gate)

Expression: Y = (A+B)'

XOR Using Basic Gates (AND, OR, NOT)

XOR = A'B + AB' (Sum of Products)

Step 1: A' = NOT(A), B' = NOT(B)

Step 2: T1 = A'.B (AND gate)

Step 3: T2 = A.B' (AND gate)

Step 4: Y = T1 + T2 (OR gate)

Expression: Y = A'B + AB' (2 NOT + 2 AND + 1 OR)

XNOR Using Basic Gates (AND, OR, NOT)

XNOR = AB + A'B' (complement of XOR)

Step 1: A' = NOT(A), B' = NOT(B)

Step 2: T1 = A.B (AND gate)

Step 3: T2 = A'.B' (AND gate)

Step 4: Y = T1 + T2 (OR gate)

Expression: Y = AB + A'B' (2 NOT + 2 AND + 1 OR)

Teacher's Signature: _____

Expt. No. _____ Date: _____

AIM: To Implement AND, OR, NOT Gates Using Universal Gates (NAND and NOR)

Universal Gates: NAND and NOR are called universal gates because any logic function can be built using only NAND gates or only NOR gates.

Using NAND Gates Only

NOT using NAND

Both inputs tied together: Y=(A.A)'=A'

Expression: Y = A'

A	Y=A'
0	1
1	0

AND using NAND

2 NAND gates: Y1=(A.B)', Y=((A.B)')'=A.B

Expression: Y = A.B

A	B	Y=A.B
0	0	0
0	1	0
1	0	0
1	1	1

OR using NAND

De Morgan: A+B=(A'.B')'. NOT A, NOT B, then NAND them.

Expression: Y = A+B (3 NAND gates)

A	B	Y=A+B
0	0	0
0	1	1
1	0	1
1	1	1

Using NOR Gates Only

NOT using NOR

Both inputs tied together: Y=(A+A)'=A'

Expression: Y = A'

A	Y=A'
0	1
1	0

Teacher's Signature: _____

Expt. No. _____ Date: _____

OR using NOR

2 NOR gates: Y1=(A+B)', Y=((A+B)')'=A+B

Expression: Y = A+B

A	B	Y=A+B
0	0	0
0	1	1
1	0	1
1	1	1

AND using NOR

De Morgan: A.B=(A'+B')'. NOT A, NOT B, then NOR them.

Expression: Y = A.B (3 NOR gates)

A	B	Y=A.B
0	0	0
0	1	0
1	0	0
1	1	1

Teacher's Signature: _____

Expt. No. _____ Date: _____

ASSIGNMENT 3

AIM: To Implement Half Adder Using Basic Gates, XOR Gates and Universal Gate

Half Adder

Theory: A Half Adder is a combinational circuit that adds two single bits and produces two outputs: Sum (S) and Carry (C). It cannot handle a carry input. Sum = A XOR B, Carry = A AND B

Truth Table:

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

1. Half Adder Using Basic Gates (AND, OR, NOT):

Sum S = A'B + AB' (XOR using basic gates)

Step 1: A' = NOT(A), B' = NOT(B)

Step 2: T1 = A'.B, T2 = A.B'

Step 3: S = T1 + T2 (OR gate)

Carry C = A.B (AND gate)

Expression: S = A XOR B, C = A.B

2. Half Adder Using XOR Gate:

Sum S = A XOR B (directly using XOR gate)

Carry C = A AND B (AND gate)

Expression: S = A ⊕ B, C = A.B

3. Half Adder Using Universal Gate (NAND only):

Carry C = (NAND(A,B))' → C = A.B

Sum S = NAND(NAND(A, NAND(A,B)), NAND(B, NAND(A,B)))

Expression: S = A ⊕ B using 4 NAND gates, C = A.B using 2 NAND gates

VHDL Code for Half Adder:

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity half_adder is
Port (A : in STD_LOGIC; -- INPUT A
B : in STD_LOGIC; -- INPUT B
S : out STD_LOGIC; -- SUM OUTPUT
C : out STD_LOGIC); -- CARRY OUTPUT
```

Teacher's Signature: _____

```
end half_adder;
architecture dataflows of half_adder is
begin
S <= A xor B; -- SUM = A XOR B
C <= A and B; -- CARRY = A AND B
end dataflows;
```

Expt. No. _____ Date: _____

AIM: To Implement Full Adder Using Basic Gates, XOR, NOR, NAND Gate

Full Adder

Theory: A Full Adder adds three bits: A, B, and Carry-in (Cin). It produces Sum (S) and Carry-out (Cout). It is used in multi-bit addition circuits. S = A XOR B XOR Cin, Cout = AB + BCin + ACin

Truth Table:

A	B	Cin	Sum (S)	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

1. Full Adder Using Basic Gates:

S = A XOR B XOR Cin (implemented using AND, OR, NOT)

Cout = AB + BCin + ACin

Two Half Adders + one OR gate = Full Adder

Expression: $S = A \oplus B \oplus Cin$, $Cout = AB + Cin(A \oplus B)$

2. Full Adder Using NAND Gates:

Full Adder can be implemented using 9 NAND gates.

XOR using NAND: 4 gates per XOR. Carry logic: additional NAND gates.

Expression: $S = A \oplus B \oplus Cin$, $Cout = AB + BCin + ACin$

3. Full Adder Using NOR Gates:

Full Adder can be implemented using 9 NOR gates.

XOR using NOR: 5 gates per XOR. Combined carry logic.

Expression: $S = A \oplus B \oplus Cin$, $Cout = AB + BCin + ACin$

VHDL Code for Full Adder:

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity full_adder is
Port (A : in STD_LOGIC; -- INPUT A
B : in STD_LOGIC; -- INPUT B
Cin : in STD_LOGIC; -- CARRY INPUT
```

Teacher's Signature: _____

```
S : out STD_LOGIC; -- SUM OUTPUT
Cout : out STD_LOGIC); -- CARRY OUTPUT
end full_adder;
architecture dataflows of full_adder is
begin
S <= A xor B xor Cin;
-- SUM = A XOR B XOR Cin
Cout <= (A and B) or (B and Cin) or (A and Cin);
-- CARRY = AB + BCin + ACin
end dataflows;
```

Expt. No. _____ Date: _____

ASSIGNMENT 4

AIM: To Design and Simulate Subtractor Using XOR/AND/NOT, Basic Gates, NAND and NOR Gates and Verify Output Using VHDL

Half Subtractor

Theory: A Half Subtractor subtracts two single bits A and B, producing Difference (D) and Borrow (Bout). D = A XOR B, Borrow = A'B

Truth Table:

A	B	Diff (D)	Borrow (B')
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

1. Using XOR, AND, NOT Gates:

D = A XOR B (XOR gate directly)

Borrow = A' AND B (NOT gate on A, then AND with B)

Expression: D = A ⊕ B, Borrow = A'B

2. Using Basic Gates (AND, OR, NOT):

D = A'B + AB' (XOR implemented with basic gates)

Step 1: A'=NOT(A), B'=NOT(B)

Step 2: T1=A'B, T2=AB'

Step 3: D = T1+T2 Borrow = A'B

Expression: D = A ⊕ B, Borrow = A'.B

3. Using NAND Gates:

D = XOR using NAND gates (4 NAND gates for XOR)

Borrow: A'B = NOT(NAND(A',B)) where A'=NAND(A,A)

Expression: D = A ⊕ B, Borrow = A'.B (using 6 NAND gates)

4. Using NOR Gates:

D = XOR using NOR gates (5 NOR gates for XOR)

Borrow = A'B = NOR based complement and AND logic

Expression: D = A ⊕ B, Borrow = A'.B

VHDL Code for Half Subtractor:

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;
```

Teacher's Signature: _____

```
Entity half_sub is
Port (A : in STD_LOGIC; -- MINUEND INPUT
B : in STD_LOGIC; -- SUBTRAHEND INPUT
D : out STD_LOGIC; -- DIFFERENCE OUTPUT
Bout : out STD_LOGIC); -- BORROW OUTPUT
end half_sub;
architecture dataflows of half_sub is
begin
D <= A xor B; -- DIFFERENCE = A XOR B
Bout <= (not A) and B; -- BORROW = A' AND B
end dataflows;
```

Full Subtractor

Theory: Full Subtractor subtracts three bits: A, B, Borrow-in (Bin). Produces Difference (D) and Borrow-out (Bout). D = A XOR B XOR Bin, Bout = A'B + A'Bin + BBin

Truth Table:

A	B	Bin	D	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

VHDL Code for Full Subtractor:

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity full_sub is
Port (A : in STD_LOGIC; -- MINUEND
B : in STD_LOGIC; -- SUBTRAHEND
Bin : in STD_LOGIC; -- BORROW INPUT
D : out STD_LOGIC; -- DIFFERENCE
Bout : out STD_LOGIC); -- BORROW OUTPUT
end full_sub;
architecture dataflows of full_sub is
begin
D <= A xor B xor Bin;
-- DIFFERENCE = A XOR B XOR Bin
Bout <= ((not A) and B) or ((not A) and Bin) or (B and Bin);
```

```
    -- BORROW = A'B + A'Bin + BBin
    end dataflows;
```

Expt. No. _____ Date: _____

ASSIGNMENT 5

AIM: To Implement 1-bit Comparator and Verify its Output Using VHDL

1-bit Comparator

Theory: A 1-bit Comparator compares two single-bit inputs A and B and produces three outputs: A>B, A=B, A<B. It is used in arithmetic and decision-making circuits.

Expressions:

A > B : AGB = A AND B' = A.B'

A = B : AEB = XNOR(A,B) = AB + A'B'

A < B : ALB = A' AND B = A'.B

Truth Table:

A	B	A > B	A = B	A < B
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

Logic Gate Implementation:

A > B: Use NOT gate on B, then AND with A → AGB = A.B'

A = B: Use XNOR gate → AEB = A XNOR B

A < B: Use NOT gate on A, then AND with B → ALB = A'.B

VHDL Code:

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity comparator_1bit is
Port (A : in STD_LOGIC; -- INPUT A
B : in STD_LOGIC; -- INPUT B
AGB : out STD_LOGIC; -- A GREATER THAN B
AEB : out STD_LOGIC; -- A EQUAL TO B
ALB : out STD_LOGIC); -- A LESS THAN B
end comparator_1bit;
architecture dataflows of comparator_1bit is
begin
AGB <= A and (not B); -- A greater than B = A.B'
AEB <= A xnor B; -- A=B : A XNOR B
ALB <= (not A) and B; -- A less than B = A'.B
end dataflows;
```

Teacher's Signature: _____

Expt. No. _____ Date: _____

AIM: To Simulate 1-bit Comparator and Verify Its Output Using VHDL

Simulation / Testbench for 1-bit Comparator:

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity tb_comparator is
end tb_comparator;

architecture tb of tb_comparator is
Component comparator_1bit
Port(A,B: in STD_LOGIC; AGB,AEB,ALB: out STD_LOGIC);
end Component;
Signal A,B,AGB,AEB,ALB: STD_LOGIC;
begin
uut: comparator_1bit port map(A,B,AGB,AEB,ALB);
process begin
A<='0'; B<='0'; wait for 100ns; -- A=B, AEB=1
A<='0'; B<='1'; wait for 100ns; -- A less B, ALB=1
A<='1'; B<='0'; wait for 100ns; -- A greater B, AGB=1
A<='1'; B<='1'; wait for 100ns; -- A=B, AEB=1
wait;
end process;
end tb;
```

Teacher's Signature: _____

Expt. No. _____ Date: _____

ASSIGNMENT 6

AIM: To Implement 2:4 Decoder Using VHDL Code With Logic Gates

2:4 Decoder

Theory: A 2:4 Decoder has 2 input lines (A1, A0) and 4 output lines (D0, D1, D2, D3) with an Enable (EN) pin. It activates exactly ONE output line based on the binary value of the inputs. Used in memory address decoding and data demultiplexing.

Expressions:

D0 = EN . A1' . A0'

D1 = EN . A1' . A0

D2 = EN . A1 . A0'

D3 = EN . A1 . A0

Truth Table:

EN	A1	A0	D0	D1	D2	D3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

Logic Gate Implementation:

4 AND gates (3-input each) + 2 NOT gates for A1', A0'

Each output: AND(EN, Ax, Ay) where Ax, Ay can be inverted

VHDL Code:

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity decoder_2to4 is
Port (A1 : in STD_LOGIC; -- INPUT MSB
A0 : in STD_LOGIC; -- INPUT LSB
EN : in STD_LOGIC; -- ENABLE
D0 : out STD_LOGIC; -- OUTPUT 0
D1 : out STD_LOGIC; -- OUTPUT 1
D2 : out STD_LOGIC; -- OUTPUT 2
D3 : out STD_LOGIC); -- OUTPUT 3
end decoder_2to4;
architecture dataflows of decoder_2to4 is
begin
D0 <= EN and (not A1) and (not A0); -- D0 = EN.A1'.A0'
```

Teacher's Signature: _____

Expt. No. _____ Date: _____

```
    D1 <= EN and (not A1) and A0; -- D1 = EN.A1'.A0
    D2 <= EN and A1 and (not A0); -- D2 = EN.A1.A0'
    D3 <= EN and A1 and A0; -- D3 = EN.A1.A0
    end dataflows;
```

Teacher's Signature: _____

Expt. No. _____ Date: _____

ASSIGNMENT 7

AIM: To Implement 4:1 MUX, 2:1 MUX, 1:4 DEMUX Using Logic Gates and 4:1 MUX Using When-Case and When-Else in VHDL

1. 2:1 Multiplexer (MUX)

Theory: A 2:1 MUX has 2 inputs (I0, I1), 1 select line (S) and 1 output (Y). Based on S, one input is forwarded to output. Expression: Y = S'I0 + SI1

S	Y
0	I0
1	I1

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity mux_2to1 is
Port (I0,I1,S : in STD_LOGIC;
Y : out STD_LOGIC);
end mux_2to1;
architecture dataflows of mux_2to1 is
begin
Y <= (I0 and (not S)) or (I1 and S);
-- Y = S'I0 + SI1
end dataflows;
```

2. 4:1 Multiplexer (MUX)

Theory: A 4:1 MUX has 4 inputs (I0-I3), 2 select lines (S1,S0) and 1 output. Expression: Y = S1'S0'I0 + S1'S0I1 + S1S0'I2 + S1S0I3

S1	S0	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity mux_4to1 is
Port (I0,I1,I2,I3,S1,S0 : in STD_LOGIC;
Y : out STD_LOGIC);
end mux_4to1;
architecture dataflows of mux_4to1 is
begin
Y <= ((not S1) and (not S0) and I0) or
```

Teacher's Signature: _____

Expt. No. _____ Date: _____

```
((not S1) and S0 and I1) or
(S1 and (not S0) and I2) or
(S1 and S0 and I3);
end dataflows;
```

Teacher's Signature: _____

Expt. No. _____ Date: _____

3. 1:4 Demultiplexer (DEMUX)

Theory: A 1:4 DEMUX routes 1 input (D) to one of 4 outputs (Y0-Y3) based on 2 select lines (S1, S0). Reverse operation of MUX.

S1	S0	Y0	Y1	Y2	Y3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity demux_1to4 is
Port (D,S1,S0 : in STD_LOGIC;
Y0,Y1,Y2,Y3 : out STD_LOGIC);
end demux_1to4;
architecture dataflows of demux_1to4 is
begin
Y0 <= D and (not S1) and (not S0); -- S1=0,S0=0
Y1 <= D and (not S1) and S0; -- S1=0,S0=1
Y2 <= D and S1 and (not S0); -- S1=1,S0=0
Y3 <= D and S1 and S0; -- S1=1,S0=1
end dataflows;
```

4. 4:1 MUX Using WHEN CASE Statement

The WITH-SELECT (case-style) concurrent statement selects output based on select signal.

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity mux_case is
Port (I0,I1,I2,I3 : in STD_LOGIC;
S : in STD_LOGIC_VECTOR(1 downto 0);
Y : out STD_LOGIC);
end mux_case;
architecture behavioral of mux_case is
begin
with S select
Y <= I0 when "00", -- S=00 → select I0
I1 when "01", -- S=01 → select I1
I2 when "10", -- S=10 → select I2
I3 when others; -- S=11 → select I3
end behavioral;
```

5. 4:1 MUX Using WHEN ELSE Statement

Teacher's Signature: _____

Expt. No. _____ Date: _____

The WHEN-ELSE concurrent statement provides conditional signal assignment.

```
Library IEEE;
Use IEEE.STD_LOGIC_1164.ALL;

Entity mux_when_else is
Port (I0,I1,I2,I3 : in STD_LOGIC;
S : in STD_LOGIC_VECTOR(1 downto 0);
Y : out STD_LOGIC);
end mux_when_else;
architecture behavioral of mux_when_else is
begin
Y <= I0 when S="00" else -- S=00 → I0
I1 when S="01" else -- S=01 → I1
I2 when S="10" else -- S=10 → I2
I3; -- S=11 → I3
end behavioral;
```

Teacher's Signature: _____